Master's Thesis Proposal

1 Review Scope and Interpretation

This review examines 13 research papers published or revised between 2017 and 2026 on secure logging, post-quantum signatures, and authentication efficiency. It establishes the background for a software-only study of adaptive checkpointing, with ML-DSA and SLH-DSA providing standardization context [14, 15]. Classical and post-quantum approaches are distinguished, and their practical trade-offs and limitations are assessed. Table 1 summarizes the methods, contributions, parameters, advantages, disadvantages, and limitations; NV indicates parameter values not verified in the accessible sources.

Abbreviations. PQ: post-quantum; ROM/QROM: classical/quantum random-oracle model; CMA: chosen-message attack; NMA: no-message attack; CP-ABE: ciphertext-policy attribute-based encryption; BFT: Byzantine fault tolerance; KEM: key-encapsulation mechanism; MPC: multiparty computation.

2 Comparative literature matrix

Table 1: Comparison of 13 selected studies relevant to adaptive post-quantum audit-log checkpointing.

Author name	Year	Model used	Main contribution	Parameters	Advantages	Disadvantages	Main limitations
Nikitin et al. [1]	2017	CHAINIAC; collectively signed skipchains and independent build witnesses.	Combines release-policy checking, source-to-binary verification, and an authenticated update history; skipchains help outdated clients validate updates and key changes.	Debian-package and PyPI workloads; witness participation and skipchain structure are design dimensions. Exact experiment configuration: NV here.	Publicly checkable release history; reduces dependence on a single release authority.	Witness coordination and reproducible-build processing add operational cost.	Software-release transparency, not streaming audit-log scheduling; the original collective-signature design is not a PQ instantiation.
Chase et al. [2]	2017	Symmetric-primitive signatures using MPC-in-the-head proofs; Picnic foundation.	Reduces the underlying proof size and studies Fiat–Shamir and Unruh transformations for non-interactive signatures.	LowMC one-way-function instantiation; alternative proof transforms; proof repetition/security settings. Complete numerical settings: NV here.	Small key pairs; diversified assumptions based on symmetric primitives.	Proof-based signatures introduce communication and computation overhead; transforms trade size for security analysis.	Signature primitive only; no log ordering, checkpoint freshness, or scheduling policy. The original paper's two transformations have different proof guarantees.
Ducas et al. [3]	2018	CRYSTALS-Dilithium; module-lattice signatures and Fiat–Shamir with aborts.	Designs an efficient signature scheme with uniform sampling and compact key representation, emphasizing secure implementation.	Module dimensions, modulus, rejection bounds, and challenge distribution determine cost/security. Original numerical profiles are not substituted with later ML-DSA profiles.	Practical signing and verification; avoids Gaussian sampling in the uniform-sampling construction.	Larger signatures than common classical schemes; rejection and arithmetic implementations still require care.	Does not itself provide log completeness, freshness, or forward security after key compromise; historical specification differs from FIPS 204.

Continued on the next page

Table 1 continued

Author name	Year	Model used	Main contribution	Parameters	Advantages	Disadvantages	Main limitations
Kim and Oh [4]	2019	FAS: forward-secure sequential aggregate signatures under a factoring assumption.	Allows third-party aggregation of consecutive signatures with constant-size public/secret keys and aggregate signatures.	Security parameter, key-evolution periods, and aggregation length; constant-size claims concern these cryptographic objects, not the retained log.	Compact authentication metadata; protects earlier periods under the scheme's forward-security model.	Requires key evolution and secure handling of old secret material.	Factoring-based security is not PQ security. Sequential aggregation is not an adaptive checkpoint controller.
Bernstein et al. [5]	2019	SPHINCS+: stateless hash-based signature framework.	Introduces FORS and tweakable hash functions, supporting unified security analysis and improved performance trade-offs.	Hash-function choice, hypertree structure, and FORS configuration; size/speed trade-offs. Exact profile values: NV here.	No persistent one-time-key usage counter; alternative to lattice assumptions.	Hash-based signatures can impose substantial signature-size and signing-time costs.	Primitive-level result, not audit-log scheduling. Stateless signing does not imply forward security or complete event capture.
Kannwischer et al. [6]	2019	pqm4 embedded testing and benchmarking framework.	Provides a common environment for testing and comparing PQ implementations on ARM Cortex-M4.	Original snapshot: 196 KiB memory, 1 MiB flash, 10 KEMs and 5 signature schemes.	Reproducible platform-level comparison; exposes resource feasibility and implementation effects.	Board-specific results require remeasurement on desktop processors.	July 2019 implementation snapshot; not evidence about current standardized variants or application-level checkpoint delay.
Putz, Menges and Pernul [7]	2019	Permissioned Exonum blockchain; off-chain logs and on-chain integrity proofs.	Implements a software-based log-auditing infrastructure and evaluates throughput, scalability, and storage; discusses grouping records to lower transaction load.	4 vCPUs/8 GB per VM; 10,000 transactions/block maximum; 1-s proposal timeout; offered load 5,000 logs/s.	No specialized logging hardware; independently checkable proofs and replicated auditing.	Consensus/storage overhead; grouping couples the verifiability of records in a group.	Original system is not PQ; already recognizes the checkpoint exposure/cost trade-off, but does not establish the proposed adaptive ML-DSA scheduling result.

Continued on the next page

Table 1 continued

Author name	Year	Model used	Main contribution	Parameters	Advantages	Disadvantages	Main limitations
Drijvers et al. [8]	2020	Pixel: pairing-based forward-secure multisignatures for consensus.	Combines key evolution with non-interactive aggregation to resist posterior corruption in blockchain signing committees.	Signature: two group elements; verification: three pairings and one exponentiation; signature size independent of signer count.	Reduces consensus authentication overhead while preserving forward security under its model.	Pairing operations and evolving keys complicate implementation.	Pairing-based construction is not PQ; multisigner block certification differs from batching records from a single log source.
Barbosa et al. [9]	2023	ROM/QROM proof repair; EasyCrypt formalization of Dilithium.	Repairs the CMA-to-NMA reduction and supplies a mechanized ROM proof; concrete analysis supports the claimed Dilithium security level.	Proof-model assumptions, oracle-query bounds, and concrete security reductions; no logging workload parameters.	Strengthens the justification for a key PQ signing primitive.	Formal reasoning requires specialist expertise and does not replace implementation testing.	Mechanized result is in the ROM; do not describe it as a fully mechanized QROM proof or an end-to-end audit-system guarantee.
Srivastava et al. [10]	2024	Mul-SAN: multivariate sanitizable signatures with a blockchain audit-log application.	Supports authorized changes to selected signed content, with signer/sanitizer accountability claims under the stated assumptions.	UOV example: $n = 160$, $m = 64$ over $GF(16)$; signature formula $2n$ field elements. Values are authors' construction settings.	Selective disclosure/editing can preserve verifiability of retained authenticated content.	Large multivariate keys; authorization of sanitization adds policy complexity.	Preprint; sanitization differs from append-only preservation. No adaptive checkpoint-delay evaluation; concrete PQ security must not be inferred from worst-case MQ hardness alone.
Kao [11]	2025/ 2026	PQ hash-and-sign evidence; hybrid signatures, legacy re-signing, and Merkle anchoring.	Defines quantum-adversary evidence notions and compares migration patterns for long-lived regulated audit records.	Illustrative archival case: $b = 4096$ records/root, $N = 10^8$ records; approximately 24,415 roots.	Directly motivates amortizing PQ authentication over batches and accounting for evidence lifetime.	Merkle proofs and batch metadata add costs beyond the root signature.	Preprint v2; case-study numbers are described as illustrative. Archival batching does not establish an online adaptive policy or unconditional global non-equivocation.

Continued on the next page

Table 1 continued

Author name	Year	Model used	Main contribution	Parameters	Advantages	Disadvantages	Main limitations
Algar-Fernandez et al. [12]	2026	Python TCP client–server orchestration of C PQ implementations.	Compares five signature families and three KEM families on commodity processors.	Two Windows 11 CPUs; ten repetitions/operation; four-byte signed message; Dilithium 2/3/5 and AES variants among evaluated profiles.	Shows that software/CPU choice affects measured PQ overhead; supports desktop-based experimentation.	Executable timings include start-up whereas DLL timings cover the routine; these boundaries affect comparisons.	Network transfer excluded; small fixed messages and limited repetitions; historical algorithm labels must not be treated as proof of FIPS conformance. No streaming-log controller.
Fugkeaw et al. [13]	2026	PQ-ABL: lattice CP-ABE, AES-GCM, and blockchain-anchored epoch Merkle roots.	Separates authorization from integrity verification; reports epoch-wide validation and standalone membership verification.	One root/epoch; $O(N)$ full-epoch validation and $O(\log N)$ membership verification. Exact epoch size, lattice settings, and hardware: NV in accessible abstract.	Integrates access control and tamper-evident cloud-forensic logging.	Requires encryption-policy management and blockchain anchoring infrastructure.	Abstract-level review only. Claimed post-validation $O(1)$ local checks are conditional; adaptive signing deadlines and end-to-end PQ ledger security need full-text assessment.

3 Research gap

Existing studies establish important components of authenticated logging, but they address different layers of the problem. Classical forward-secure aggregation and collective histories reduce authentication overhead or improve compromise resilience [4, 8, 1], while PQ signature research supplies and analyzes quantum-resistant signing primitives [3, 5, 9]. Crucially, neither batching nor PQ audit evidence is itself a new contribution: Putz et al. discuss grouped records and checkpoint exposure [7], Kao explicitly studies PQ Merkle-root anchoring [11], and recent PQ-ABL work uses epoch-based authenticated logging [13]. Within the evidence reviewed here, however, a controlled demonstration of an online checkpoint policy that adapts to changing arrivals and signing service time while satisfying an explicit authentication-delay target remains insufficiently established. The defensible research opportunity is therefore to design and evaluate *deadline-aware adaptive checkpointing for a software-only ML-DSA audit logger*, rather than introduce another hash-and-sign combination. The policy should be compared with per-record signing, fixed-count, fixed-time, and combined count-or-time baselines under identical cryptographic settings and steady, sparse, and bursty workloads. Evaluation should jointly measure signing CPU cost, complete authentication-storage overhead, tail delay, deadline violations, and the number of unanchored records, including overload and restart conditions. Security analysis must distinguish modification detection for externally acknowledged prefixes from unverifiable unsigned suffixes, missing source events, and signer compromise. This is a bounded research gap supported by the selected literature, not an exhaustive claim that no adaptive authenticated-logging method exists; the closest recent systems require full-text comparison before asserting priority.

4 Proposed thesis positioning

Working title: *Deadline-Aware Adaptive Checkpointing for Post-Quantum Signed Audit Logs.*

Research question: Can a workload-aware checkpoint policy reduce signing CPU cost and authentication bytes per event relative to well-tuned fixed policies, without increasing deadline violations or weakening the explicitly defined integrity guarantees?

The intended contribution is a checkpoint-scheduling mechanism and a reproducible evaluation, not a new signature primitive. Use one fixed ML-DSA parameter set for primary comparisons, with a separately reported sensitivity study if resources permit [14]. A laptop implementation, a log generator/replay script, a signer, and a separate verifier are sufficient for the experimental scope. Physical IoT devices, an IoT simulator, and a blockchain are not required. A verifier process on the same laptop represents logical separation only; it does not establish protection against compromise of that laptop.

4.1 Minimum comparison protocol

- **Baselines:** per-record signing; fixed record-count batching; fixed time-interval batching; and a count-or-time rule that closes a batch when either threshold is reached. Tune fixed-policy parameters on separate development workloads.
- **Fairness:** keep signature parameters, record encoding, commitment structure, checkpoint metadata, acknowledgement path, and hardware identical. Randomize execution order and replay matched workloads across methods.
- **Measurements:** CPU time per event, signatures per 1,000 events, authentication bytes per event including metadata/proofs, peak memory, p95/p99 delay, deadline-violation rate, and peak unanchored-event count. Report repeated-run uncertainty rather than only the best

run.

- **Stress and integrity tests:** sparse arrivals, bursts, service slowdown, verifier unavailability, restart, modification, replay, reordering, and truncation relative to retained trusted checkpoints. Deliberately test the limits of detecting an unsigned suffix.
- **Ablation and interpretation:** compare the adaptive policy with its fixed-policy counterpart using the same deadline guard. A null result is informative: the controller may add overhead without improving the cost–delay frontier.

5 Evidence and version notes

The matrix distinguishes historical algorithms from standards and avoids transferring benchmark values between them. FAS uses the journal issue year (2019), although its DOI contains 2018. Kao is one paper, counted once, with initial submission in 2025 and a revised 2026 title/version. The pqm4, Mul-SAN, and Kao references identify the specific public research versions used. PQ-ABL's unverified implementation details are explicitly marked rather than reconstructed. Before submission, obtain its full text and check additional nearby work on adaptive authenticated batching; a selective review cannot certify novelty.

References

[1] K. Nikitin, E. Kokoris-Kogias, P. Jovanovic, N. Gailly, L. Gasser, I. Khoffi, J. Cappos, and B. Ford, "CHAINIAC: Proactive software-update transparency via collectively signed skipchains and verified builds," in *26th USENIX Security Symposium*, 2017, pp. 1271–1287. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/nikitin.

[2] M. Chase, D. Derler, S. Goldfeder, C. Orlandi, S. Ramacher, C. Rechberger, D. Slamanig, and G. Zaverucha, "Post-quantum zero-knowledge and signatures from symmetric-key primitives," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. https://www.microsoft.com/en-us/research/publication/post-quantum-zero-knowledge-and-signatures-from-symmetric-key-primitives/.

[3] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018. https://doi.org/10.13154/tches.v2018.i1.238-268.

[4] J. Kim and H. Oh, "FAS: Forward secure sequential aggregate signatures for secure logging," *Information Sciences*, vol. 471, pp. 115–131, 2019. https://doi.org/10.1016/j.ins.2018.08.044.

[5] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019. https://doi.org/10.1145/3319535.3363229. Author manuscript: https://eprint.iacr.org/2019/1086.

[6] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4," *Cryptology ePrint Archive*, Paper 2019/844, 2019. Preprint; July 2019 snapshot. https://eprint.iacr.org/2019/844.

[7] B. Putz, F. Menges, and G. Pernul, "A secure and auditable logging infrastructure based on a permissioned blockchain," *Computers & Security*, vol. 87, article 101602, 2019. https://doi.org/10.1016/j.cose.2019.101602. Accepted manuscript: https://epub.uni-regensburg.de/40693/.

[8] M. Drijvers, S. Gorbunov, G. Neven, and H. Wee, "Pixel: Multi-signatures for consensus," in *29th USENIX Security Symposium*, 2020, pp. 2093–2110. https://www.usenix.org/conference/usenixsecurity20/presentation/drijvers.

[9] M. Barbosa, G. Barthe, C. Doczkal, J. Don, S. Fehr, B. Grégoire, Y.-H. Huang, A. Hülsing, Y. Lee, and X. Wu, "Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium," in *Advances in Cryptology—CRYPTO 2023*, 2023. Author manuscript: https://eprint.iacr.org/2023/246.

[10] V. Srivastava, P. Baidya, S. Mesnager, D. Roy, and S. K. Debnath, "Blockchain-envisioned post-quantum secure sanitizable signature for audit logs management," *arXiv preprint arXiv:2312.16322v2*, 25 March 2024; first submitted 26 December 2023. https://arxiv.org/abs/2312.16322v2.

[11] L. Kao, "Post-quantum-resilient audit evidence for long-lived regulated systems: Security models, migration patterns, and case study," *arXiv preprint arXiv:2512.00110v2*, 8 February 2026; first submitted 27 November 2025. https://arxiv.org/abs/2512.00110v2.

[12] J. Algar-Fernandez, A. Villacís-Vanegas, Y. Amaro-Aular, and M.-D. Cano, "Benchmarking post-quantum signatures and KEMs on general-purpose CPUs using a TCP client–server testbed," *Computers*, vol. 15, no. 2, article 116, 2026. https://doi.org/10.3390/computers15020116.

[13] S. Fugkeaw, N. Jitkhajornwanich, P. Mongkolchukaie, N. Rouengsamai, and N. Pornpinyamad, "Optimized post-quantum attribute-based logging for real-time and tamper-evident cloud forensics," *IEEE Transactions on Cloud Computing*, vol. 14, no. 2, pp. 1098–1114, 2026. https://doi.org/10.1109/TCC.2026.3681599. Publisher record: https://ieeexplore.ieee.org/abstract/document/11475768.

[14] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, 13 August 2024. https://doi.org/10.6028/NIST.FIPS.204. Current publication and errata record: https://csrc.nist.gov/pubs/fips/204/final.

[15] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard*, FIPS 205, 13 August 2024. https://doi.org/10.6028/NIST.FIPS.205.